UADE

Procesos de Desarrollo de Software

CLASE 1 – METODOLOGÍAS DEL DESARROLLO, CLASES

Objetivos de la cursada

- Entender el proceso de desarrollo de software
- Modelado de clases
- Comprender y aplicar los patrones GRASP y principios SOLID
- Comprender patrones de arquitectura. MVC, Clean Architecture, CQRS.
- Entender los tipos de patrones de diseño y su aplicación.
- Entender la deuda técnica. Code Smells.

Agenda de la clase

- Aprobación de la cursada
- Análisis y Diseño
- Metodologías del desarrollo
- Proceso Unificado
- UML
- Diagrama de clases, POO.

Aprobación de la cursada

- 2 (**dos**) examenes parciales
  - 1(**una**) única instancia de recuperatorio, aplicable a uno de los dos parciales.
- 1 (**un**) final
  - posibilidad de rendir final adelantado si ha aprobado ambos parciales.
- La materia **NO** es promocionable.
- Ver cronograma para las fechas de examen.

Herramientas

- StarUML para modelado de diagramas.
- Eclipse o IntelliJ IDEA como IDE de trabajo.
- Entregas por MS Teams (**NO** se aceptan entregas atrasadas)

Análisis y Diseño

- **Análisis**
- Investigar el problema.
- Determinar los requisitos (¿o requerimientos?)

- **Diseño**
- Diagramar la solución conceptual.
- Implementar la solución.

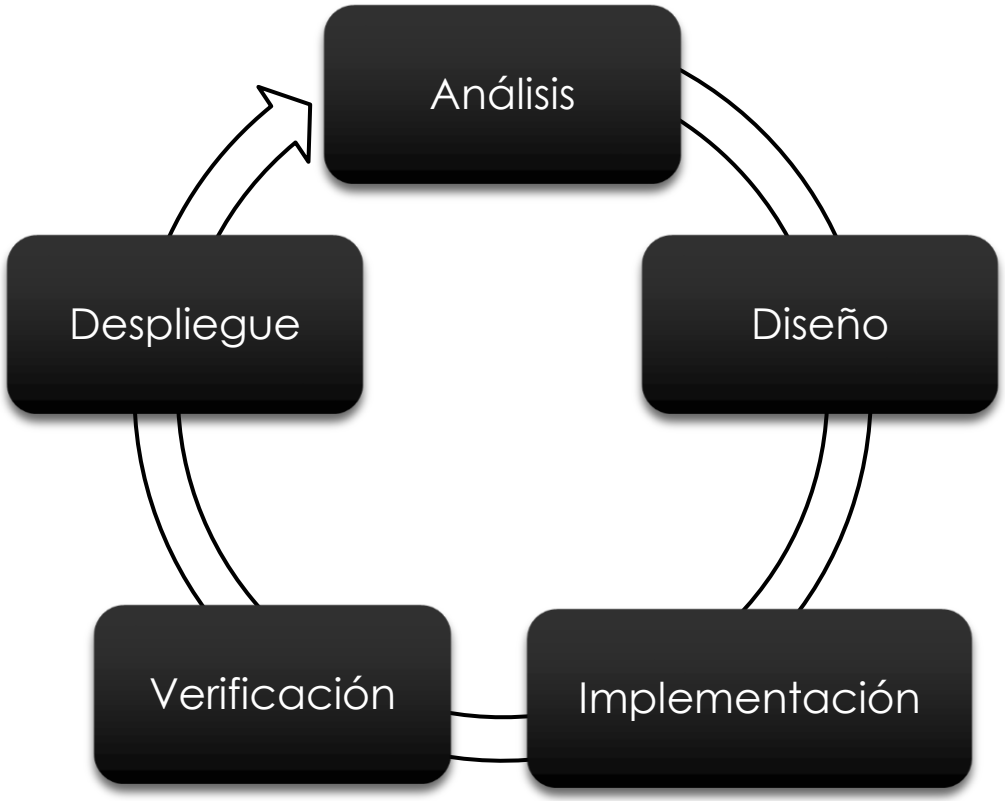
Metodologías de desarrollo Tradicional

- El desarrollo está ordenado en etapas.
- No puedes avanzar a una etapa hasta completar la anterior.



Metodologías de desarrollo Iterativo e Incremental

- Cada iteración involucra todas las actividades de desarrollo.
- En cada iteración se produce un incremento del software.



Diferencias: tradicional vs iterativo

	Desarrollo en Cascada	Desarrollo Iterativo e Incremental
Tipo de proyecto	Estructurado	Flexible
Organizado en forma de	Un gran proyecto	A base de proyectos pequeños
Cuándo usarlos	Situaciones donde el cambio es poco común	Proyectos que requieren mejoras continuas o MVPs (Minimum Viable Product)
Postura frente a los requisitos	Requisitos claramente definidos por adelantado	Los requisitos van a evolucionar y cambiar

Proceso Unificado

- Proceso de desarrollo software para la construcción de sistemas orientados a objetos.
- Se ha convertido en un proceso de desarrollo de software de gran éxito para la construcción de sistemas orientados a objetos.
- El Proceso Unificado de **Rational** (**R**UP) es un refinamiento actualizado y detallado del UP, que es más general.

RUP / Características

Guiado por Casos de Uso

Un Caso de Uso es una funcionalidad del sistema que proporciona al usuario un resultado importante.

Centrado en Arquitecturas

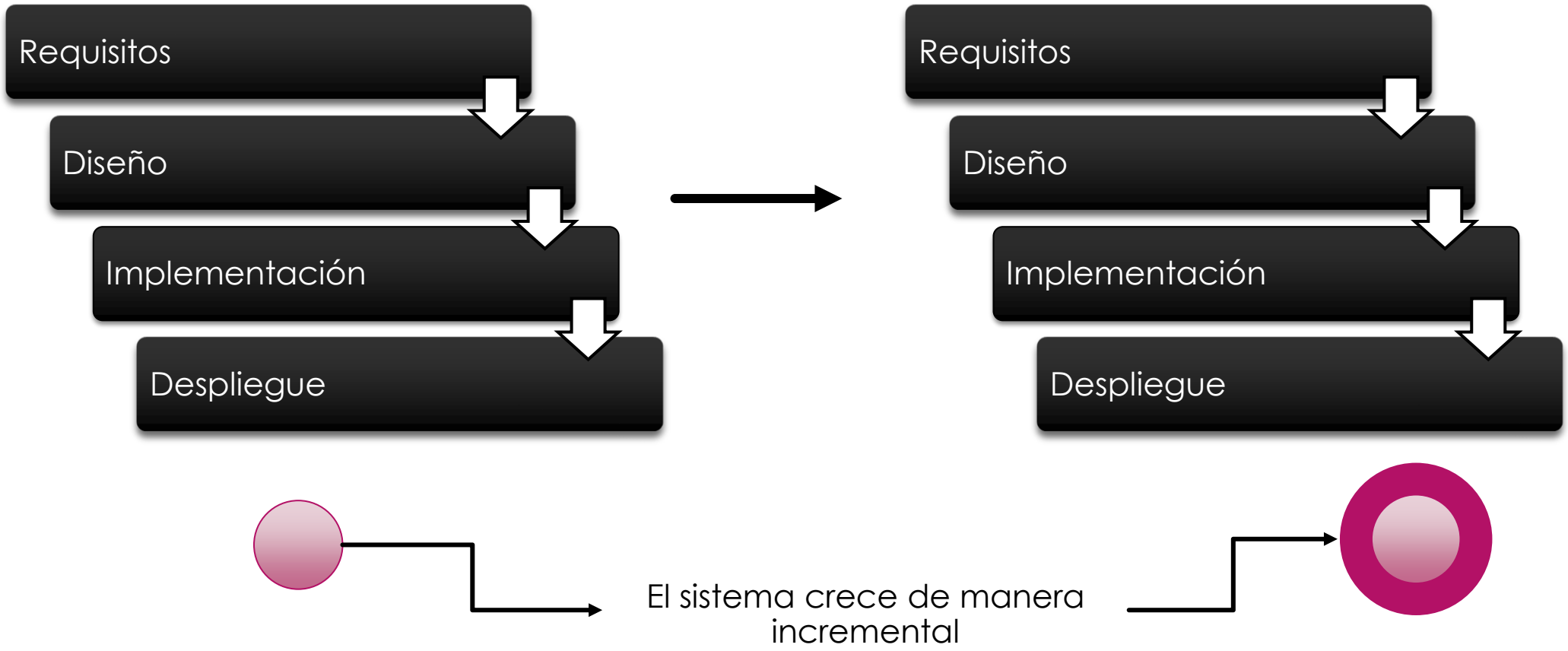
Plataforma en la que funcionará el software.

Desarrollo Iterativo e Incremental

Divide el trabajo en partes pequeñas y manejables.

Cada mini proyecto es una iteración que resulta en un incremento.

RUP / Desarrollo iterativo e incremental



RUP / Beneficios del desarrollo iterativo

- **Mitigación** temprana de **riesgos altos**.
- Progreso visible en las primeras etapas.
- Una temprana **retroalimentación**, compromiso de los usuarios y adaptación.
- **Gestión** de la **complejidad**: tareas complejas se dividen en problemas más simples.
- El conocimiento adquirido durante una iteración se puede utilizar metódicamente para mejorar el propio proceso de desarrollo.

RUP / Las **4** P's

Proyecto

- Su finalidad es producir un producto de software

Producto

- Son todos los artefactos que se crean durante el ciclo de vida de un proyecto.

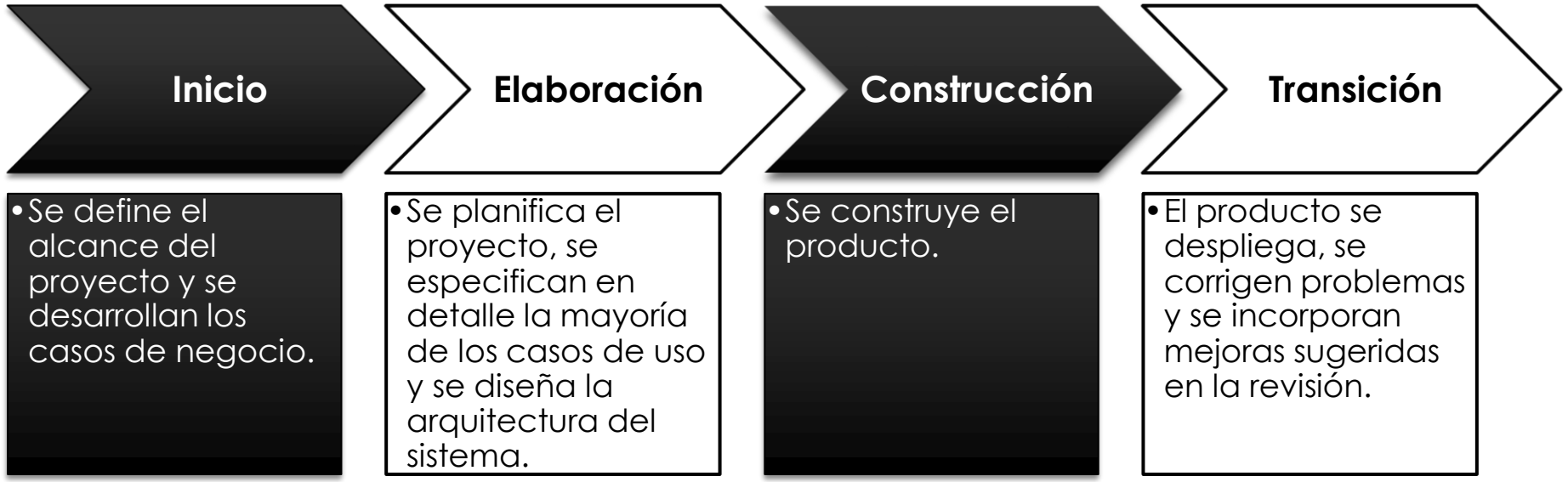
Proceso

- Mediante el cual los proyectos producen productos de manera efectiva.

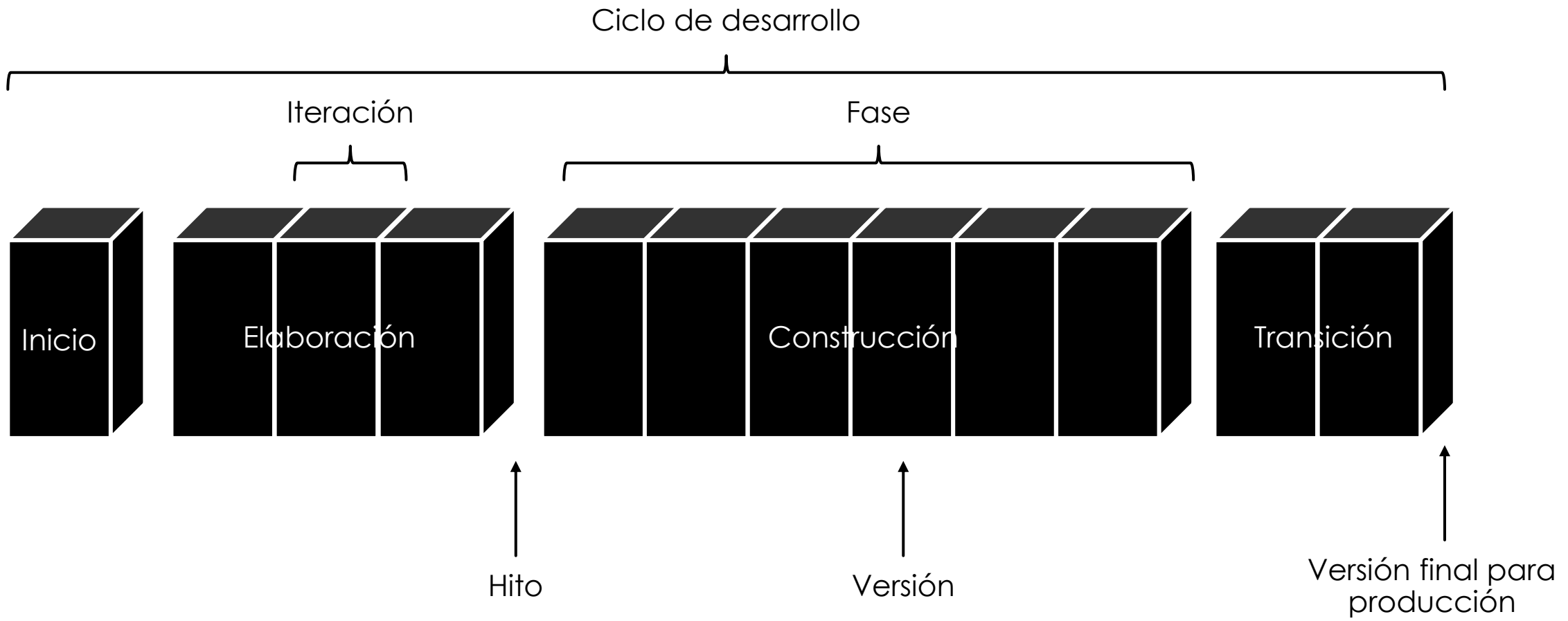
Personas

- La dinámica interpersonal del equipo influye en los logros del proyecto.

RUP / Las cuatro fases del SDLC



RUP / Ciclo de vida



RUP / Glosario

Iteración

Es una secuencia de actividades con un plan establecido y unos criterios de evaluación, cuyo resultado es un hito.

Hito

Punto de control para los participantes del desarrollo de software.

Disciplina

Conjunto de actividades y artefactos relacionados en un área determinada.

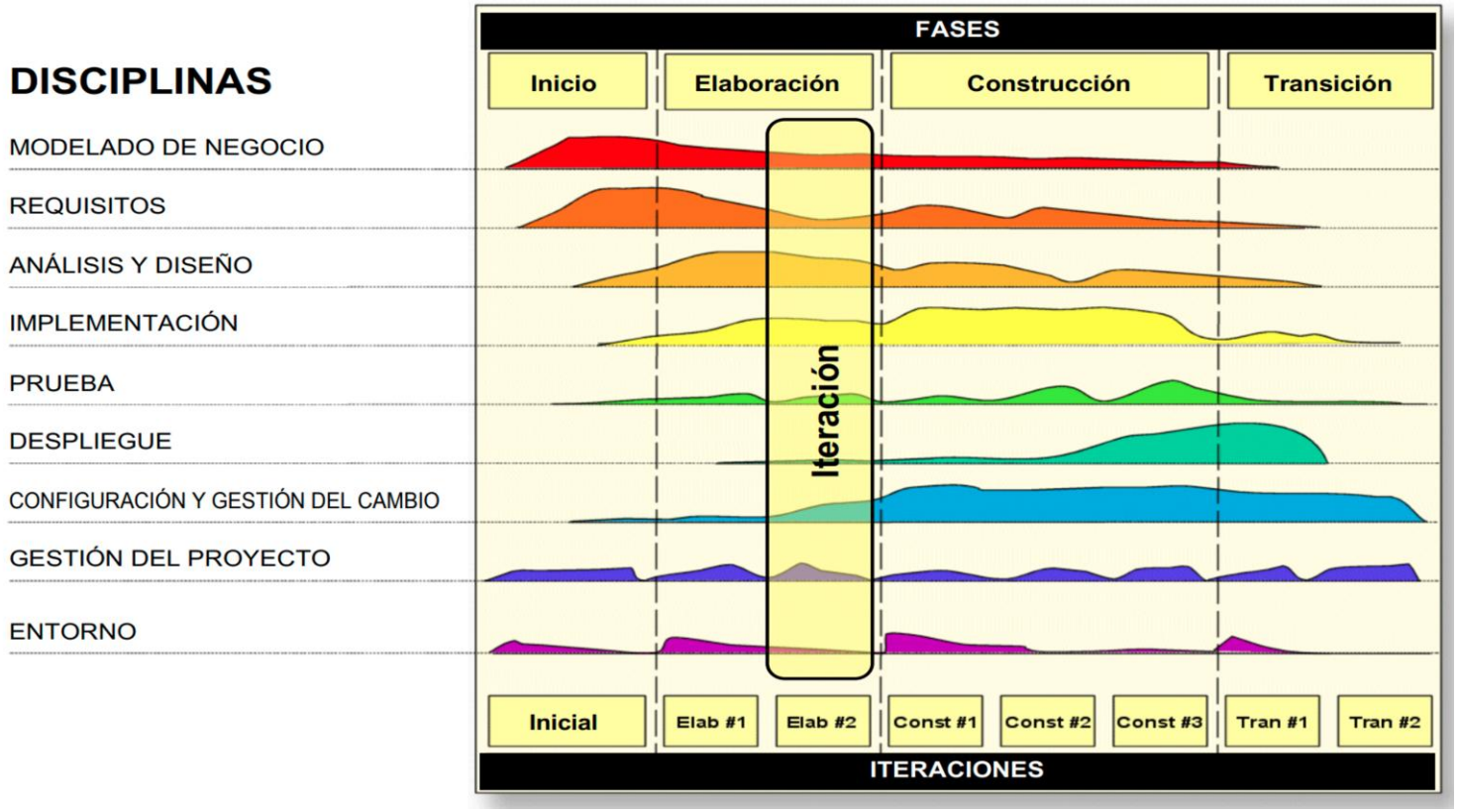
Artefacto

Cualquier tipo de información creada, producida, cambiada o utilizada por los stakeholders en el desarrollo del sistema.

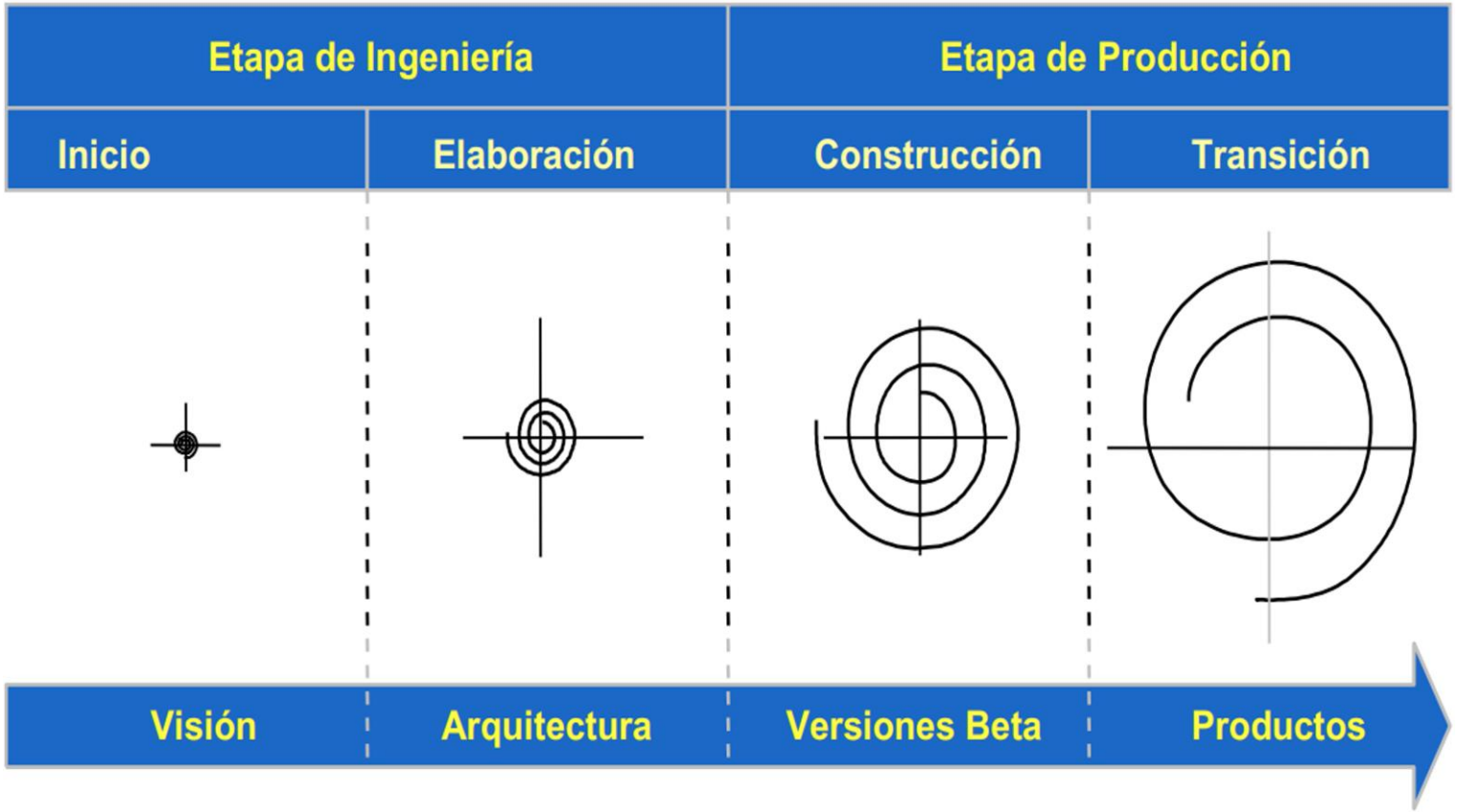
Stakeholders

Personas involucradas en el desarrollo de software.

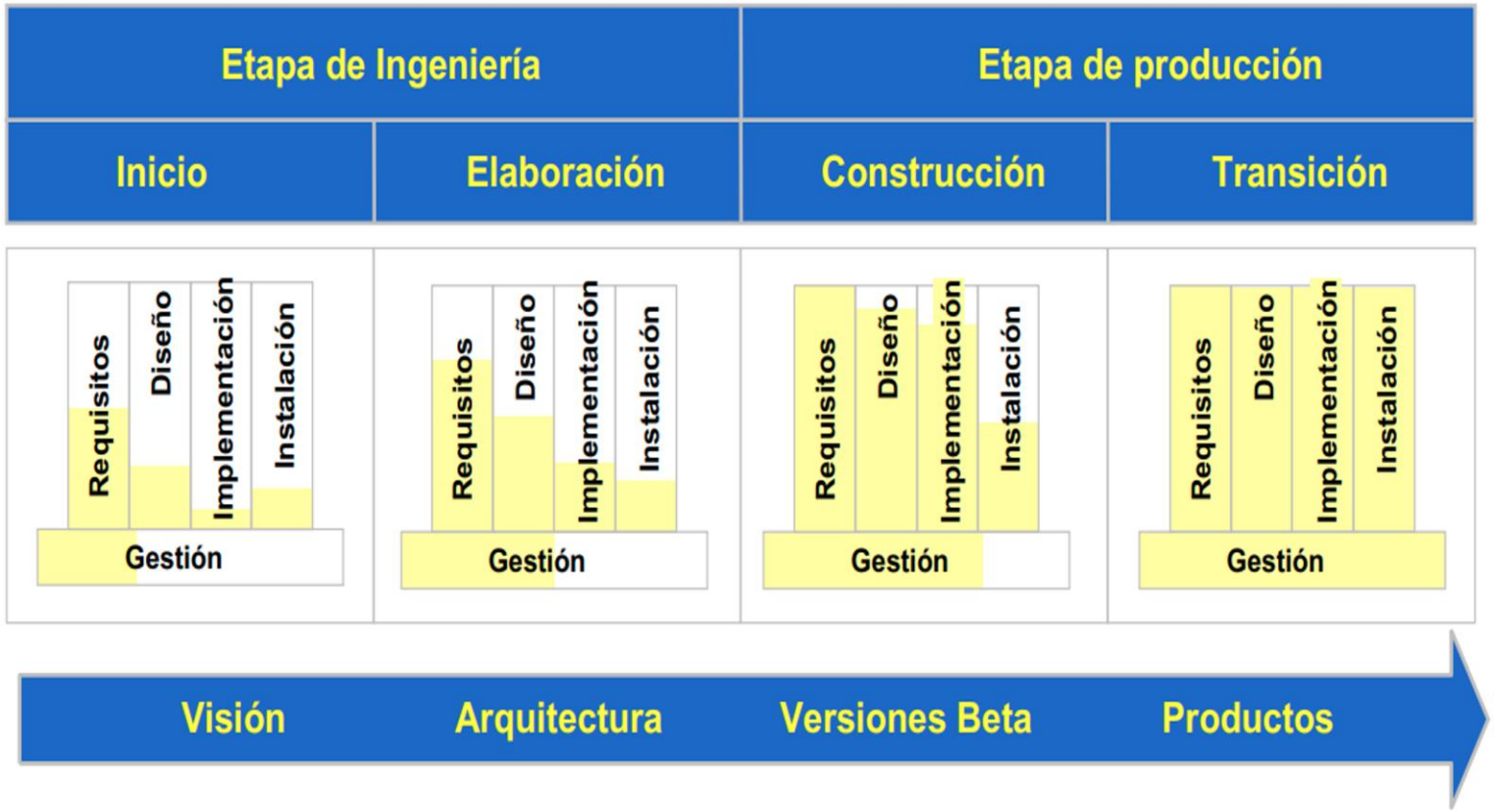
RUP / Ciclo de vida



RUP / Iteratividad



RUP / Iteratividad



UML

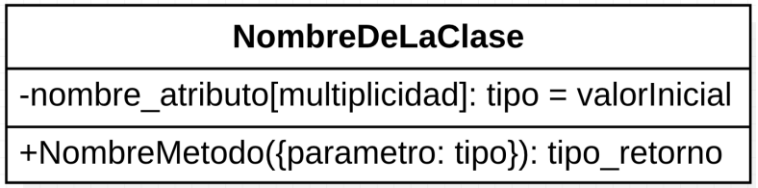
- Es un lenguaje universal para especificar, visualizar, construir y documentar los artefactos de los sistemas software.
- Brinda a arquitectos de sistemas, ingenieros y desarrolladores de software las herramientas para el análisis, el diseño y la implementación de sistemas basados en software, así como para el modelado de procesos de negocios y similares.

UML / Diagramas y vistas

Tipo de Vista	Diagrama
Comportamiento	Casos de Uso
	Actividades
	Maquina de estados
Interacción	Secuencia
	Colaboración
	Tiempo
	Interacción Global
Estructura	Paquetes
	Clases
	Objetos
	Estructura Compuesta
	Componentes
	Despliegue

Clases

- Una clase es una plantilla o molde.
- Define atributos y métodos.
- Permite crear objetos.



+ Pública

Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.

- Privada

Representa que se puede acceder al atributo o función únicamente desde la misma clase.

Protegida

Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas).

Ejemplo de clase en Java

```java
class Persona {

    String nombre;
    int edad;

    void saludar(){
        System.out.println("Hola soy " + nombre);
    }

}
```

¿Qué es un objeto?

- Un objeto es una instancia de una clase.
- Representa una entidad concreta.
- Los objetos tienen estado propio.
- Todo objeto cuenta con tres características que lo diferencia del resto:
  - **Identidad**
  - **Comportamiento**
  - **Estado Interno**

Crear un objeto en Java

```java
Persona p = new Persona();

p.nombre = "Juan";
p.edad = 30;

p.saludar();
```

Pilares de la programación orientada a objetos

- Organizar el código fuente, y
- re-usar código fuente en similares contextos.



Abstracción

- Permite modelar las características relevantes.
- Oculta los detalles innecesarios.
- Los atributos y métodos de una clase deben ser relevantes para el dominio que estamos modelando.

Ejemplo abstracción

- En un sistema podemos modelar un **Auto**.
- En la realidad un auto tiene:
  - motor
  - pistones
  - sistema de inyección
  - combustible
  - sensores
  - transmisión
  - etc.

```
class Auto {

    private int velocidad;

    public void acelerar() {
        velocidad += 10;
    }

    public void frenar() {
        velocidad -= 10;
    }

    public int getVelocidad() {
        return velocidad;
    }
}
```

Encapsulamiento

- Protege los datos de un objeto.
- Se accede a ellos mediante métodos.
- El encapsulamiento nos permite controlar quien puede ver y utilizar los distintos módulos internos de nuestro sistema.
- En términos de clases, con el encapsulamiento definimos el acceso a los miembros de la clase.

Ejemplo encapsulamiento

```java
class Auto {

    public int velocidad;

}

public class Main {

    public static void main(String[] args) {

        Auto auto = new Auto();

        auto.velocidad = 200;
        auto.velocidad = -50;  // ¡valor inválido!

        System.out.println(auto.velocidad);

    }

}
```



```java
class Auto {

    private int velocidad;

    public void acelerar() {
        velocidad += 10;
    }

    public void frenar() {
        velocidad -= 10;
    }

    public int getVelocidad() {
        return velocidad;
    }

}

public class Main {
    public static void main(String[] args) {
        Auto auto = new Auto();

        auto.acelerar();
        auto.acelerar();

        System.out.println(auto.getVelocidad());
    }

}
```

Herencia

- La herencia es una relación especial entre dos clases, la clase **base** y la clase **derivada**, en donde la clase **derivada** obtiene la habilidad de utilizar ciertas **propiedades** y **funcionalidades** de la clase **base**, incluso **pudiendo sustituir** funcionalidad de la clase **base**. La idea es que la clase **derivada** "**hereda**" algunas de las características de la clase **base**.
- Permite crear nuevas clases a partir de otras.
- Facilita reutilización de código.

Ejemplo herencia

```java
class Vehiculo {

    protected int velocidad;

    public void acelerar() {
        velocidad += 10;
        System.out.println("El vehículo acelera.");
    }

    public void mostrarVelocidad() {
        System.out.println("Velocidad: " + velocidad);
    }

}
```

```java
class Auto extends Vehiculo {

    @Override
    public void acelerar() {
        velocidad += 20;
        System.out.println("El auto acelera más rápido.");
    }

    public void abrirBaul() {
        System.out.println("El baúl del auto está abierto.");
    }
}
```

```java
class Moto extends Vehiculo {

    @Override
    public void acelerar() {
        velocidad += 15;
        System.out.println("La moto acelera.");
    }

}
```

Polimorfismo

- Permite tratar diferentes objetos de forma uniforme.
- Una operación puede comportarse diferente.
- Polimorfismo significa de **muchas formas**.
- Polimorfismo significa que distintos objetos pueden responder de forma diferente al mismo mensaje.

Ejemplo polimorfismo

```java
class Vehiculo {

    protected int velocidad;

    public void acelerar() {
        velocidad += 10;
        System.out.println("El vehículo acelera.");
    }

    public void mostrarVelocidad() {
        System.out.println("Velocidad: " + velocidad);
    }
}
```

```java
class Auto extends Vehiculo {

    @Override
    public void acelerar() {
        velocidad += 20;
        System.out.println("El auto acelera más rápido.");
    }

}
```

```java
class Moto extends Vehiculo {

    @Override
    public void acelerar() {
        velocidad += 15;
        System.out.println("La moto acelera.");
    }

}
```

```java
public class Main {

    public static void main(String[] args) {

        Vehiculo v1 = new Auto();
        Vehiculo v2 = new Moto();

        v1.acelerar(); // Llama al método acelerar() de Auto
        v2.acelerar(); // Llama al método acelerar() de Moto

    }
}
```

- Por qué puede ser peligroso sobrescribir un método ya implementado?

Repasando: Biblioteca Universitaria

La biblioteca de la universidad desea modelar los **materiales que pueden prestarse**.

Todos los materiales tienen:

- título
- año de publicación

Existen dos tipos de materiales:

- **Libro** → préstamo por **15 días**
- **Revista** → préstamo por **7 días**

Repasando: Biblioteca Universitaria

```java
abstract class MaterialBiblioteca {

    private String titulo;
    private int anioPublicacion;

    public MaterialBiblioteca(String titulo, int anioPublicacion) {
        this.titulo = titulo;
        this.anioPublicacion = anioPublicacion;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getAnioPublicacion() {
        return anioPublicacion;
    }

    public abstract int calcularDuracionPrestamo();
}
```

Repasando: Biblioteca Universitaria

```java
class Libro extends MaterialBiblioteca {

    public Libro(String titulo, int anioPublicacion) {
        super(titulo, anioPublicacion);
    }

    @Override
    public int calcularDuracionPrestamo() {
        return 15;
    }
}
```

```java
class Revista extends MaterialBiblioteca {

    public Revista(String titulo, int anioPublicacion) {
        super(titulo, anioPublicacion);
    }

    @Override
    public int calcularDuracionPrestamo() {
        return 7;
    }
}
```

```java
public class Main {

    public static void main(String[] args) {

        MaterialBiblioteca m1 = new Libro("Clean Code", 2008);
        MaterialBiblioteca m2 = new Revista("National Geographic", 2023);

        System.out.println(m1.getTitulo() + " - dias de prestamo: " + m1.calcularDuracionPrestamo());
        System.out.println(m2.getTitulo() + " - dias de prestamo: " + m2.calcularDuracionPrestamo());
    }
}
```

Gracias!

- 1. PDS Ejercicio 1
- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

